

LEARNING TO WALK WITH PPO



Thomas Schmied

June 2024

drl@ml.jku.at

LIT AI LAB, Institute for Machine Learning

Slides: Marius Constantin Dinu, Thomas Schmied

Recap & Overview

Previously:

- Imitation Learning → DAgger, discrete control
- Value-based RL → Deep Q-networks (DQN), off-policy, discrete control

Today:

- Policy-based RL → Proximal Policy Optimization (PPO), on-policy, policy-gradient, actor-critic, continuous control

Notation & Terminology

- **MDP** $M = (\mathcal{S}, \mathcal{A}, r, p, \gamma)$, state $s \in \mathcal{S}$, actions $a \in \mathcal{A}$, reward $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, discount factor $\gamma \in [0, 1)$, state-transition probability $p(s' | s, a)$, $p : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$
- **Reward function**: $r(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$ and analogously for a trajectory $\tau \sim (s_0, a_0, s_1, a_1, \dots, s_t, a_t)$ according to $r(\tau)$
- **Return**: $G_t = \sum_{k=t}^{\infty} \gamma^k R_{t+k+1}$ discounted future reward
- **Policy** $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$: With stochastic policy $\pi(a | s)$ for agent behavior
- **State-value** function: $V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$ when following policy π
- **Action-value** function: $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$ following policy π
- **Performance measure**: $J(\theta)$

Policy-based vs value-based comparison

Policy-based (today):

- most are model-free and on-policy
- often favourable for continuous action spaces
- may converge faster (to local minima)
- gradient step in direction of maximizing the return / direct action selection

Value-based (previously):

- most are off-policy
- more sample efficient than policy-based methods
- computes expectations over value function of states / not used for action selection (action is indirectly retrieved by maximum over Q values)

Policy Gradient

- Learning policy π_θ directly based on scalar performance measure $J(\theta)$ [Sutton and Barto, 1998]:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta)$$

- Derivation of Policy Gradient for on-policy (using log-derivative trick)

$$\nabla_\theta \mathbb{E}_{\pi_\theta} [r(\tau)] = \mathbb{E} [\nabla_\theta \log \pi_\theta(\tau) r(\tau)]$$

- Term $\mathbb{E} [\nabla_\theta \log \pi_\theta(\tau)]$ is log likelihood.
- It measures likelihood of current trajectory given policy π_θ .
- Gradient $\nabla_\theta \mathbb{E}_{\pi_\theta} [r(\tau)]$ increases likelihood of high return trajectories, and decreases likelihood of trajectories with negative returns.

REINFORCE I

Algorithm REINFORCE (on-policy)

Initialize policy $\pi_{\theta}(\cdot)$ with random weights, define learning rate α

repeat:

Generate episode $\{s_0, a_0, R_1, s_1, \dots, s_{T-1}, a_{T-1}, R_T, s_T\} \sim \pi_{\theta}(\cdot)$

$$\nabla_{\theta} J(\theta) \approx \sum_i (\sum_t \nabla_{\theta} \log \pi_{\theta}(a_{t,i} \mid s_{t,i})) (\sum_t r(s_{t,i}, a_{t,i}))$$

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

Intuition:

- good actions are made more likely
- bad actions are made less likely
- “trial and error” approach

REINFORCE II

■ Issues:

- ☐ **High variance** / noisy gradients → unstable learning
- ☐ **Sample inefficient** → cannot reuse samples directly
- ☐ Easily collapses to bad solutions → Problems with **exploration**

■ Tricks:

- ☐ Causality Trick
- ☐ Baseline, action-critic
- ☐ Importance sampling, trust region constraints
- ☐ Maximum Entropy

Reducing Variance I - Causality Trick

- **First**, actions at time t' (present) cannot affect reward at time t when $t < t'$ (past), allowing us to use “reward-to-go” [Levine, 2020]:

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim (p(s), \pi_{\theta}(\cdot | s))} [\nabla_{\theta} \log \pi_{\theta}(\tau) r(\tau)] \\ &\approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{t,i} | s_{t,i}) \left(\sum_{t=1}^T r(s_{t,i}, a_{t,i}) \right) \\ &\approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{t,i} | s_{t,i}) \left(\sum_{\substack{t'=t}}^T r(s_{t',i}, a_{t',i}) \right) \\ &\approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{t,i} | s_{t,i}) \hat{Q}^{\pi}(s_{t,i}, a_{t,i})\end{aligned}$$

Reducing Variance II - Baseline

- **Second**, we can subtract constant b from our reward function that is **independent of actions** to reduce variance:

$$\begin{aligned}\nabla_{\theta} J(\theta) &\approx \frac{1}{n} \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(\tau) [r(\tau) - b] \\ \mathbb{E}_{\pi_{\theta}(\tau)} [\nabla_{\theta} \log \pi_{\theta}(\tau) b] &= \int \pi_{\theta}(\tau) \frac{\nabla_{\theta} \pi_{\theta}(\tau)}{\pi_{\theta}(\tau)} b \, d\tau \\ &= \int \nabla_{\theta} \pi_{\theta}(\tau) b \, d\tau \\ &= b \nabla_{\theta} \int \pi_{\theta}(\tau) \, d\tau = b \nabla_{\theta} 1 = 0\end{aligned}$$

Actor-Critic Methods I

- We want to increase “good” action and decrease “bad” actions → reduce variance by subtracting a **baseline**.
- Typically using critic as baseline → **Actor-critic**
- Baseline can have various forms:

$$\nabla J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla \log \pi_{\theta}(a | s) G_t]$$

REINFORCE

$$= \mathbb{E}_{\pi_{\theta}} [\nabla \log \pi_{\theta}(a | s) Q^{\pi}(s, a)]$$

Q Actor-Critic

$$= \mathbb{E}_{\pi_{\theta}} [\nabla \log \pi_{\theta}(a | s) A^{\pi}(s, a)]$$

Advantage Actor-Critic

[Fragkiadaki and Mitchell, 2018], [Weng, 2018]

Actor-Critic Methods II

- **Critic** estimates action-value $Q(s, a)$ or state-value $V(s)$.
- **Actor** updates policy distribution in direction suggested by Critic.
- Preferable to use **advantage function** for Critic, because smaller / more stable gradients:

$$Q(s_t, a_t) = \mathbb{E}[R_{t+1} + \gamma V(s_{t+1})]$$

$$\begin{aligned} A(s_t, a_t) &= Q(s_t, a_t) - V(s_t) \\ &= R_{t+1} + \gamma V(s_{t+1}) - V(s_t) \end{aligned}$$

only requiring one estimator for the value function $V(s)$.

- **Intuitively:** How good is a specific action compared to average action at s_t .

Importance Sampling

- **Recall:** on-policy learning is sample-inefficient $\rightarrow$ old collected samples not directly reusable after update $\rightarrow$ distribution shift
- **Importance sampling** used to estimate value functions for a **policy** π with samples collected previously from **old policy** $\bar{\pi}$.

$$\mathbb{E}_{a_t \sim \bar{\pi}_{\theta_{\text{old}}}(\cdot), s_t \sim p(s)} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(\prod_{t'=1}^t \frac{\pi_{\theta}(a_{t'} | s_{t'})}{\bar{\pi}_{\theta_{\text{old}}}(a_{t'} | s_{t'})} \right) \left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) \right) \right]$$

- If both policies are **close enough**, allows us to reuse old samples
- **BUT** issues if π_{θ} moves too far away

Trust Region Methods I - TRPO

- We can define new objective with **constraint** (= trust region) to ensure that new policy remains ϵ -close to old policy [Schulman et al., 2015]:

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\bar{\pi}_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t^{\pi} - \beta \text{KL}[\bar{\pi}_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)] \right]$$

- where $\text{KL}[\cdot]$ is Kullback–Leibler divergence between old/new policy, $\hat{A}_t$ an estimator of advantage function, β a hyperparameter.
- The further it moves away, the more it is penalized.
- **Trust region** helps to not take over-optimistic actions that hurt training.

Trust Region Methods II - PPO

- Because β is difficult to tune, and computing $\text{KL}[\cdot]$ is expensive, PPO suggests a “Clipped Surrogate Objective” [Schulman et al., 2018]:

$$\hat{\mathbb{E}}_t \left[\min \left(\varphi_t(\theta) \hat{A}_t, \text{clip}(\varphi_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^\pi \right) \right]$$
$$\varphi_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\bar{\pi}_{\theta_{\text{old}}}(a_t \mid s_t)}$$

with ϵ as a hyperparameter.

- First term is same as TRPO surrogate objective without the constraint, second term creates lower bound (i.e., pessimistic bound) on unclipped objective.

Exploration: Maximum Entropy

- **Recall:** Need to encourage **exploration** and help prevent early convergence to sub-optimal policies.
- We can augment the standard maximum RL objective with an entropy regularization term:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (A^{\pi}(s_t, a_t) + \beta \nabla_{\theta} \mathcal{H}(\pi_{\theta}(\cdot | s_t)))]$$

Proximal Policy Optimization - PPO

Algorithm PPO, Actor-Critic

Initialize actor $\pi_\theta(\cdot)$, critic $\nu_\omega(\cdot)$ with random weights, create empty buffer $\mathcal{D}$, learning rates $\alpha_{1..2}$

for $t = 0$ **to** T **do**:

Generate and store n transition tuples from timestep t

$$\mathcal{D} \leftarrow \mathcal{D} \cup \{s_i, a_i, R_{i+1}, s_{i+1}, \log \bar{\pi}_{\theta_{\text{old}}}(\cdot | s_i) | \bar{\pi}_{\theta_{\text{old}}}\}_{i=t}^{t+n}$$

for k update steps **do**:

Sample a batch $\mathcal{B}$ of transitions from the buffer $\mathcal{D}$

$$(s_j, a_j, R_{j+1}, s_{j+1}, \bar{\pi}_{\theta_{\text{old}}}(\cdot | s_j)) \sim \mathcal{B}_{\mathcal{D}}$$

Compute target function $v_{\text{target}} = R_{j+1} + \gamma \nu_\omega(s_{j+1})$

Compute advantage function $\hat{A}_j = v_{\text{target}} - \nu_\omega(s_j)$

Compute importance sampling ratio $\hat{\varphi}_j(\theta) = \log \frac{\pi_\theta(a_j | s_j)}{\bar{\pi}_{\theta_{\text{old}}}(a_j | s_j)}$

$$\mathcal{L}^{\text{CLIP}}(\theta, \omega) = -\hat{\mathbb{E}}_j \left[\min \left(\hat{\varphi}_j(\theta) \hat{A}_j(\omega), \text{clip}(\hat{\varphi}_j(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_j(\omega) \right) \right]$$

$$\mathcal{L}^{\text{VAL}}(\omega) = \|v_{\text{target}} - \nu_\omega(s_j)\|^2$$

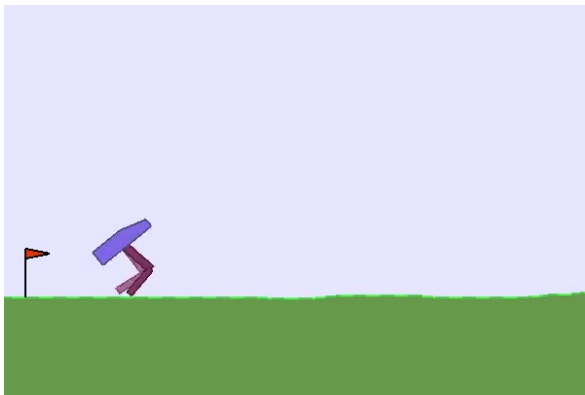
$$\mathcal{L}^{\text{ENT}}(\theta) = -\mathcal{H}(\pi_\theta(\cdot | s_j))$$

$$\mathcal{L}(\theta, \omega) = \mathcal{L}^{\text{VAL}}(\omega) + \mathcal{L}^{\text{CLIP}}(\theta, \omega) + \mathcal{L}^{\text{ENT}}(\theta)$$

$$\theta \leftarrow \theta - \alpha_1 \nabla_\theta \mathcal{L}(\theta, \omega), \omega \leftarrow \omega - \alpha_2 \nabla_\omega \mathcal{L}(\theta, \omega)$$

reset buffer $\mathcal{D}$

Learning to Walk Again



BipedalWalker Video, Gym Webpage

Submission

- Submission server: <https://apps.ml.jku.at/challenge/>
- Exercise provided on Moodle
- Train actor $\pi_{\theta} = \mathcal{N}(\mu, \sigma)$ and sample from distribution
- Export only your trained actor as ONNX file
- Upload ONNX file to submission server
- Submission closes on **23:59, June 11th.**
- Moodle submission stays open 24h after the challenge closes
- Please upload a **zip file** named `k<studentid>.zip` containing:
 - ☐ `code.ipynb`
 - ☐ `model.onnx`
 - ☐ `report.pdf`

Evaluation

- ≤ -50 : 0 Pts.
- > -50 : 10 Pts.
- > 50 : 20 Pts.
- > 150 : 30 Pts.
- > 250 : 40 Pts.

Recommendations

- You should see some progress in about 15 min
- Handle high variance in your reward (range < -100 and $> +300$)
- Optionally impl. GAE [[Schulman et al., 2016](#)]
- Try shallow network architectures
- Play around with hyperparameters after you have something working
- Change only one thing at a time and try out an experiment
- The critic has to learn faster than the actor

Hints - PPO

- Again: **DON'T use Transformers on this problem!** Deep models will not help.
- Agent is not getting better: Talk with your colleagues and post on forum. Please start soon.
- To get you started: $\text{total} = 1000$, $\text{epochs} = 4$, $\text{min_transitions} = 2048$, $\text{batch_size} = 128$, $\text{lr} = 1e - 4$, $\gamma = 0.99$
- Check-out evaluation script for hints on action selection.

REFERENCES



References I

[Fragkiadaki and Mitchell, 2018] Fragkiadaki, K. and Mitchell, T. (2018).
Deep reinforcement learning and control.

[Levine, 2020] Levine, S. (2020).
Policy gradients CS 285 | Deep Reinforcement Learning.

[Schulman et al., 2015] Schulman, J., Levine, S., Moritz, P., Jordan, M. I., and
Abbeel, P. (2015).
Trust region policy optimization.

*In 32st International Conference on Machine Learning (ICML), volume 37 of
Proceedings of Machine Learning Research, pages 1889–1897. PMLR.*

References II

- [Schulman et al., 2016] Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. (2016).
High-dimensional continuous control using generalized advantage estimation.
In Proceedings of the International Conference on Learning Representations (ICLR).
- [Schulman et al., 2018] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2018).
Proximal policy optimization algorithms.
ArXiv, 1707.06347.

References III

[Sutton and Barto, 1998] Sutton, R. S. and Barto, A. G. (1998).

Reinforcement Learning - An Introduction.

MIT Press, Cambridge, MA.

[Weng, 2018] Weng, L. (2018).

Policy gradient algorithms.

Technical report, Lil'Log.